MINI-PROJECT REPORT

Unit 2: Network Models and Protocols

Name: Sanskriti Santoshkumar Uttam
PRN: 2125UCSF1114
Department : Cybersecurity
Course: Network Protocol Configuration (Fortinet)
Academic Year: 2026–27

---

1. OBJECTIVE

The objective of this mini-project is to capture and analyse live network traffic using Wireshark, an open-source packet analyser. The project focuses on an unencrypted HTTP session with the website neverssl.com.

The project demonstrates how to identify the TCP three-way handshake (SYN, SYN-ACK, ACK) and the HTTP GET request that follows it. It also identifies the source and destination IP addresses and port numbers used at the Transport layer.

---

2. SOFTWARE USED

Software: Wireshark (open-source)

Wireshark is a network protocol analyser that captures packets on a network interface and displays their contents in detail. It supports display filters, which make it possible to isolate specific protocols, such as HTTP or TCP, from a large capture.

Browser used: Google Chrome
Capture interface: Wi-Fi
Test website: http://neverssl.com (a plain HTTP site, so its traffic is not encrypted)

---

3. PROCEDURE

Step 1: Installing Wireshark
Wireshark was downloaded from the official website and installed with the default options, including the packet-capture driver.

Step 2: Starting the Packet Capture
Wireshark was opened and the primary network interface (Wi-Fi) was selected. The capture was started by clicking the Start Capturing Packets button.

Step 3: Generating HTTP Traffic
While the capture was running, a web browser was opened and the following address was visited:

http://neverssl.com

This site uses HTTP instead of HTTPS, so the request and response are visible in plain text.

Step 4: Stopping and Saving the Capture
The capture was stopped after the page loaded. The packets were saved as:

Unit_2_Wireshark_HTTP_Capture.pcapng

The capture contained 1761 packets in total.

Step 5: Filtering HTTP Traffic
The display filter http was applied to show only HTTP packets.

Figure 1: HTTP Filter Applied

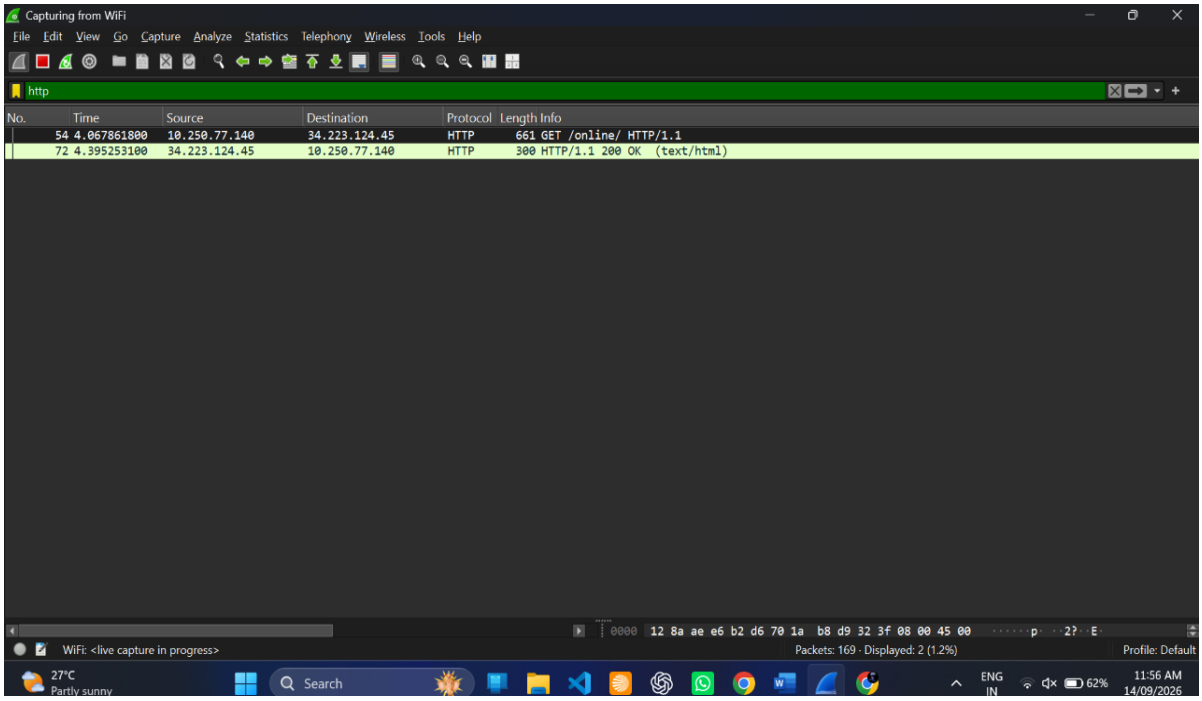


Figure 1: Wireshark display filtered with "http".

Step 6: Locating the TCP Three-Way Handshake

The filter tcp.flags.syn == 1 was applied to find the connection set-up packets. The SYN packet and the SYN-ACK packet of the connection to neverssl.com (34.223.124.45, port 80) were identified.

Figure 2: TCP SYN and SYN-ACK Packets

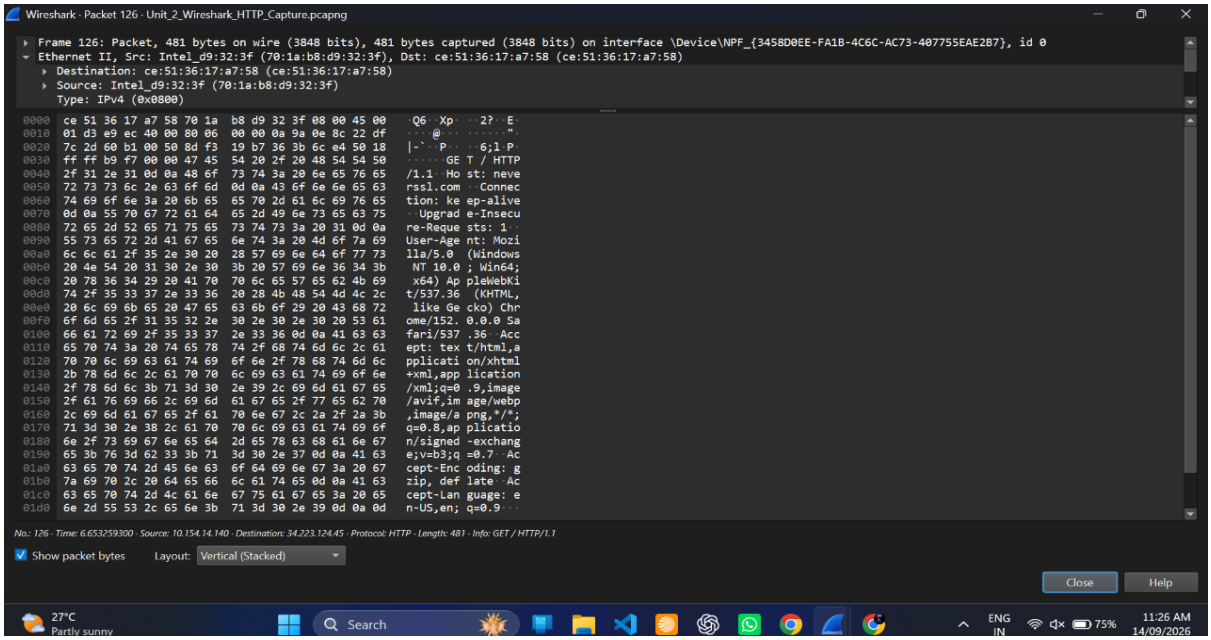


Figure 2: Packets with the SYN flag set, showing the handshake with 34.223.124.45 on port 80.

Step 7: Locating the HTTP GET Request

The HTTP GET request was found in Frame 126. Its Frame, Ethernet, IP and TCP details were examined in the packet details pane.

Figure 3: Frame, Ethernet and IP Details of Frame 126

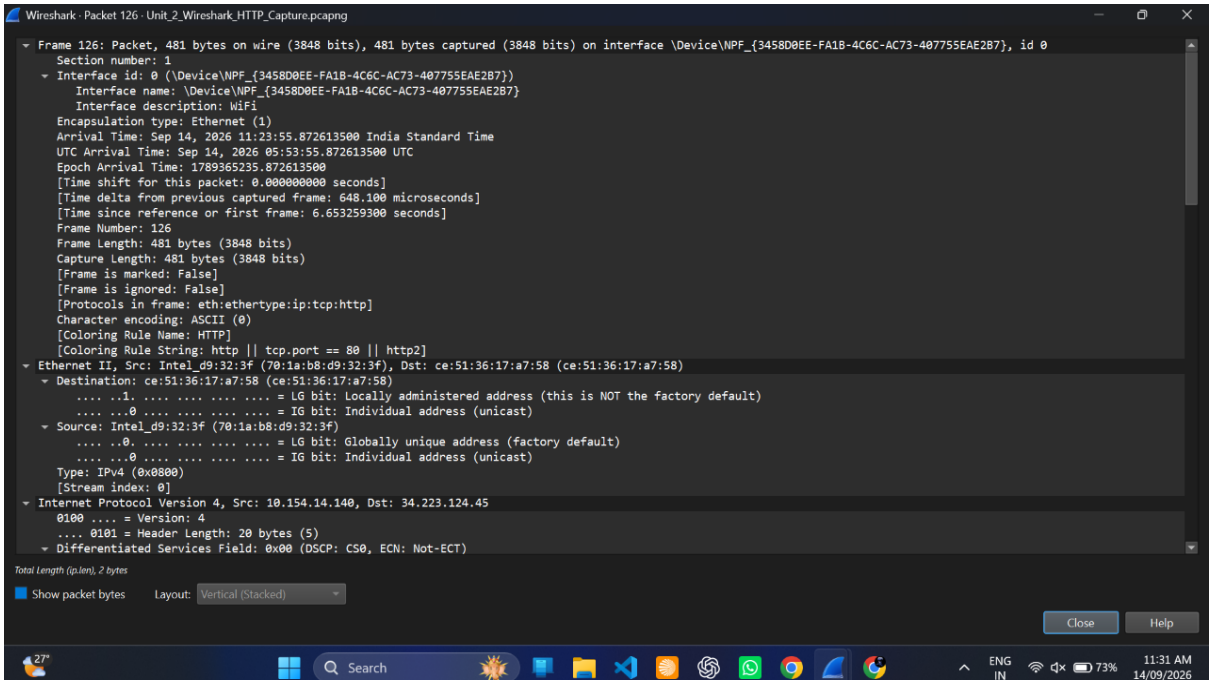


Figure 3: Frame 126 details showing source IP 10.154.14.140 and destination IP 34.223.124.45.

Figure 4: TCP Details of Frame 126

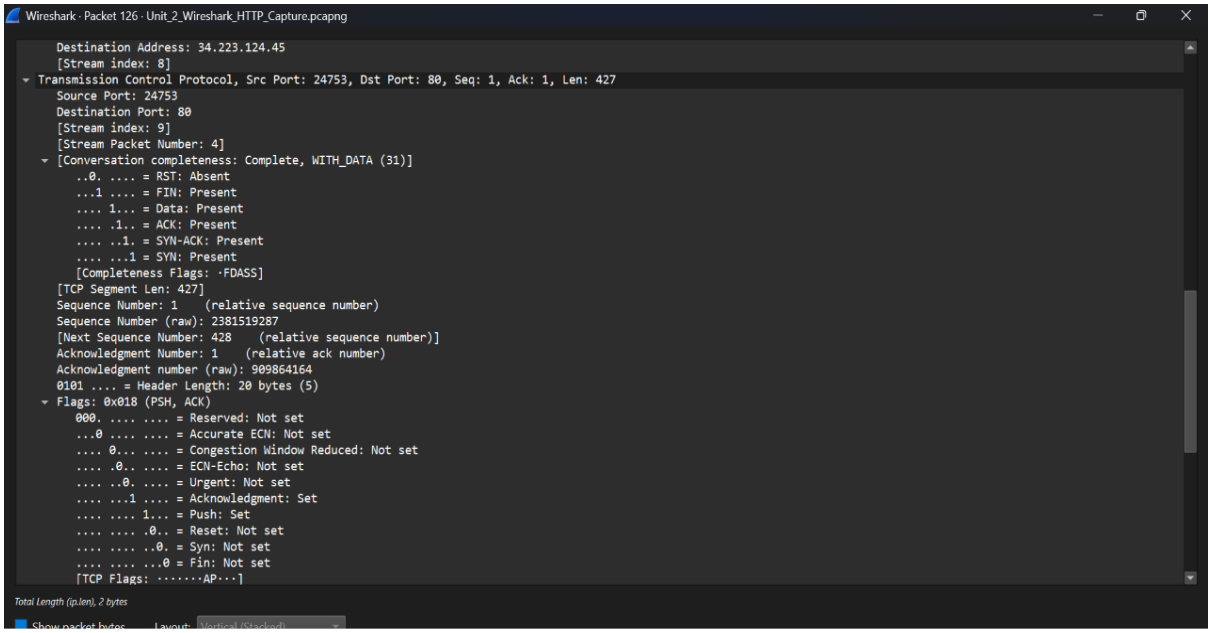


Figure 4: TCP header of Frame 126 showing source port 24753, destination port 80 and flags PSH, ACK.

Figure 5: HTTP GET Request in Packet Bytes

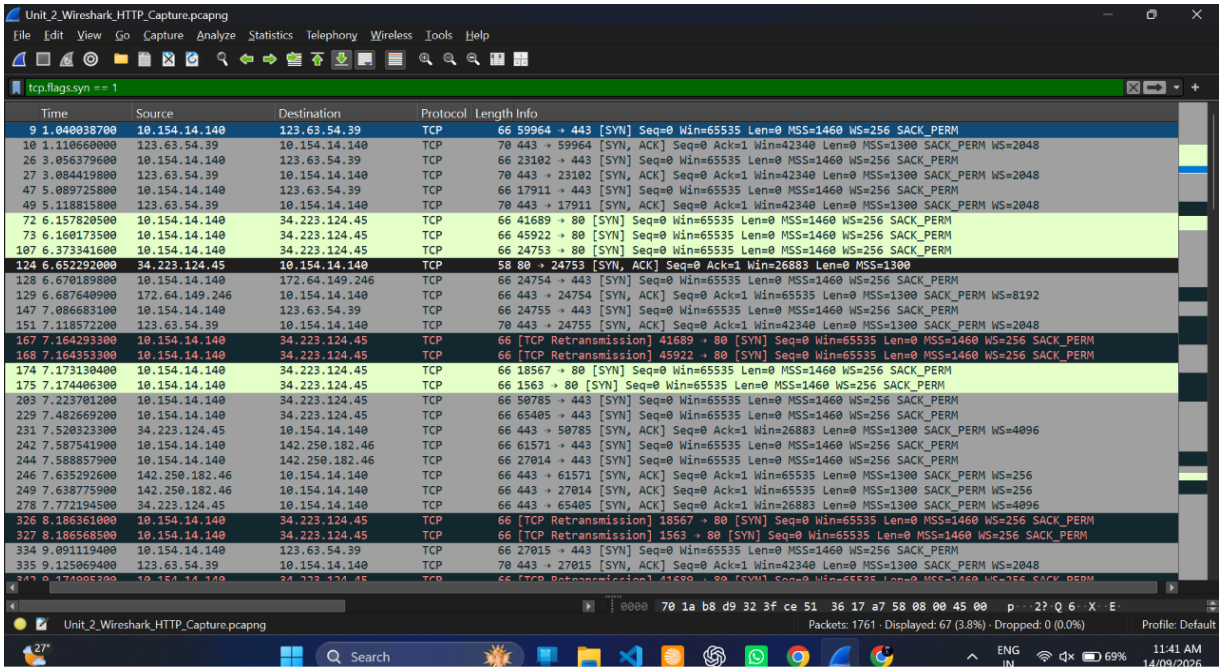


Figure 5: Packet bytes showing "GET / HTTP/1.1" and "Host: neverssl.com".

---

4. TCP THREE-WAY HANDSHAKE

Before any HTTP data is sent, TCP sets up a reliable connection using three packets.

Step	Frame	Direction	TCP Flags	Details
1. SYN	107	10.154.14.140:24753 → 34.223.124.45:80	SYN	Seq=0, Win=65535, MSS=1460
2. SYN-ACK	124	34.223.124.45:80 → 10.154.14.140:24753	SYN, ACK	Seq=0, Ack=1, Win=26883, MSS=1300

3. ACK	125	10.154.14.140:24753 → 34.223.124.45:80	ACK	Seq=1, Ack=1

- In the first step, the client sends a SYN to request a connection.
- In the second step, the server accepts the request with a SYN-ACK.
- In the third step, the client confirms with an ACK. The connection is then established and data can be exchanged.

The SYN was sent at about 6.373 s and the SYN-ACK arrived at about 6.652 s, so the round trip to the server took roughly 0.28 s.

---

5. HTTP GET REQUEST

Immediately after the handshake, the browser sent the HTTP GET request.

Parameter	Value
Frame Number	126
Arrival Time	Sep 14, 2026, 11:23:55 IST
Protocol	HTTP
HTTP Method	GET
Request URI	/
HTTP Version	HTTP/1.1
Host	neverssl.com
Frame Length	481 bytes
TCP Payload Length	427 bytes
TCP Flags	PSH, ACK

Sequence / Acknowledgment Number	1 / 1 (relative)

The request header also contained Connection: keep-alive and a User-Agent field identifying Chrome on Windows 10. Because HTTP is not encrypted, all of this was readable in the packet bytes (Figure 5).

---

6. TRANSPORT LAYER ANALYSIS

Parameter	Value
Source IP Address	10.154.14.140 (client)
Destination IP Address	34.223.124.45 (neverssl.com server)
Source TCP Port	24753
Destination TCP Port	80
Transport Protocol	TCP
Application Protocol	HTTP
Source MAC Address	70:1a:b8:d9:32:3f
Destination MAC Address	ce:51:36:17:a7:58

---

7. SUMMARY

The client at IP address 10.154.14.140 opened a TCP connection to the web server at 34.223.124.45. The client used source port 24753, a temporary port chosen by the operating system for this one connection. The server listened on destination port 80, the well-known port for HTTP.

The connection was set up with the three-way handshake: SYN in Frame 107, SYN-ACK in Frame 124 and ACK in Frame 125. The browser then sent the HTTP GET request for the home page (“/”) of neverssl.com in Frame 126, with the PSH and ACK flags set.

Because the website uses HTTP and not HTTPS, the request was sent in plain text. This shows why unencrypted traffic can be read by anyone who captures it on the network.

The capture also showed that the browser opened several connections at once, and some SYN packets were retransmitted. The connection from port 24753 was the one analysed in this report.

---

8. RESULT

The HTTP traffic to neverssl.com was successfully captured and analysed using Wireshark. The TCP three-way handshake and the HTTP GET request were identified, and the source and destination IP addresses and port numbers were recorded.

---

9. CONCLUSION

This mini-project provided practical experience in capturing and analysing network traffic with Wireshark. It demonstrated the use of display filters, the TCP three-way handshake, the structure of an HTTP GET request, and the role of IP addresses and port numbers at the Network and Transport layers.
